# Claim Complexity Algorithm

*Project Requirements and Specifications*

Penser North America




**WSU Student Team**

Kevin Herrera, Brian Leung, Ishaan Joshi

Sep 30th, 2026

## Table of Contents

**I. Introduction** ..... **2**

**II. Bios** ..... **2**
**II.1. Kevin Herrera – Team Lead** ..... **2**
**II.2. Brian Leung** ..... **2**
**II.3. Ishaan Joshi** ..... **2**

**III. System Requirements Specification** ..... **3**

**III.1. Use Cases** ..... **3**
**UC-1: Submit Claim** ..... **3**
**UC-2: View Claim Analysis** ..... **4**
**UC-3: API Claim Analysis** ..... **4**
**UC-4: Retrain Models** ..... **5**

**III.2. Functional Requirements** ..... **5**
**III.2.1. Claim Analysis and Complexity Model** ..... **5**
**III.2.2. User Interface and Integration** ..... **7**

**III.3. Non-Functional Requirements** ..... **7**

**III.4. User Stories** ..... **8**
**III.4.1. User Stories** ..... **8**
**III.4.2. Traceability Matrix** ..... **10**

**III.5. Standards** ..... **11**

**IV. System Evolution** ..... **12**

**V. Glossary** ..... **12**

**VI. References** ..... **13**

## I. Introduction

This project will develop an artificial intelligence system for analyzing insurance claims. The system will produce two primary outputs: a claim complexity score, representing the expected difficulty of processing a claim, and an expected payout, estimating the potential financial cost of the claim.

The system will consist of machine learning models, a web-based interface, and an API. Customers and insurance employees will use the web interface, while insurance companies will be able to integrate the system into larger applications through the API. The system is intended to provide data-driven decision support rather than replace the judgment of insurance professionals.

## II. Bios

### II.1 Kevin Herrera – Team Lead

Kevin Herrera is a Computer Science student at Washington State University with experience developing full-stack and mobile applications. His technical background includes React Native, Expo, Next.js, TypeScript, Hugging Face, PostgreSQL, and REST APIs, with a focus on authentication, database design, and role-based access control. Through independent software development, Kevin has built applications for invoicing, tournament management, and field-service operations, including software deployed for live business testing. He also has experience developing AI-driven applications, including a WSU hackathon project that received the Best Use of Presage award. For this project, Kevin contributes experience in application development, backend systems, and integrating machine learning functionality into practical software systems.

### II.2 Brian Leung

Brian Leung is a Computer Science student at Washington State University with a minor in Mathematics and experience in machine learning, data science, and distributed systems. His technical background includes Python, C/C++, SQL, PyTorch, Hugging Face, NumPy, pandas, Neo4j, and Apache Spark. Brian has developed machine learning projects involving natural language processing and neural networks, including a tweet sentiment classifier using parameter-efficient fine-tuning and a handwritten digit classifier built from scratch. He has also worked with large-scale data analytics, developing a distributed Amazon co-purchasing analysis system using Neo4j and Apache Spark. As a Teaching Assistant for programming and data structures courses, he has experience helping students apply programming, data structures, and algorithm analysis concepts. For this project, Brian contributes experience in machine learning, data analysis, algorithm development, and software engineering.

### II.3 Ishaan Joshi

Ishaan Joshi is a Software Engineering student at Washington State University pursuing a minor in Mathematics, with experience in software development, formal methods, and artificial intelligence. His technical background includes C/C++, Python, finite automata, GitHub/GitLab, and experience with AI and machine learning. Ishaan has worked on projects involving accessible navigation software, application development, and symbolic graph analysis, including a BDD-based symbolic model checker for evaluating graph reachability and quantified first-order logic statements. His coursework includes algorithms, software engineering, data structures, automata and formal languages, probability and statistics, linear algebra, and graph theory. For

this project, Ishaan contributes experience in software development, algorithms, mathematical modeling, and formal methods.

## II. System Requirements Specification

The system requirements describe the functionality and properties necessary to provide claim analysis to customers, insurance employees, and corporate users.

### III.1. Use Cases

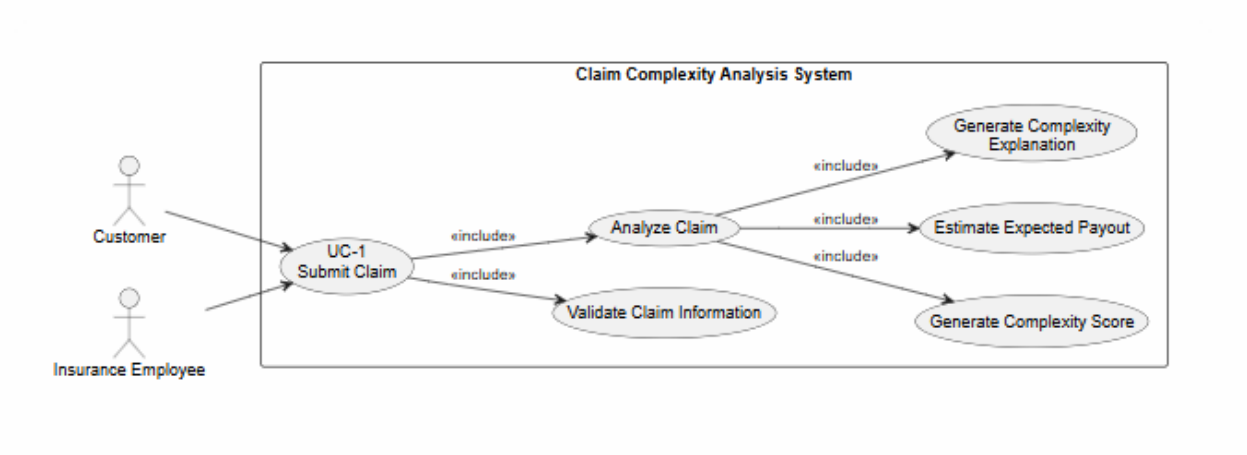



**Figure 1.** UC-1 UML diagram

**UC-1: Submit Claim**

**Actor:** Customer or Insurance Employee

An authenticated user submits claim information through the web interface. The system validates the information and submits it for analysis. The analysis produces a complexity score, expected payout, and explanation of the complexity score.

**Related Requirements:** FR-1, FR-2, FR-3, FR-4, FR-5, NFR-1

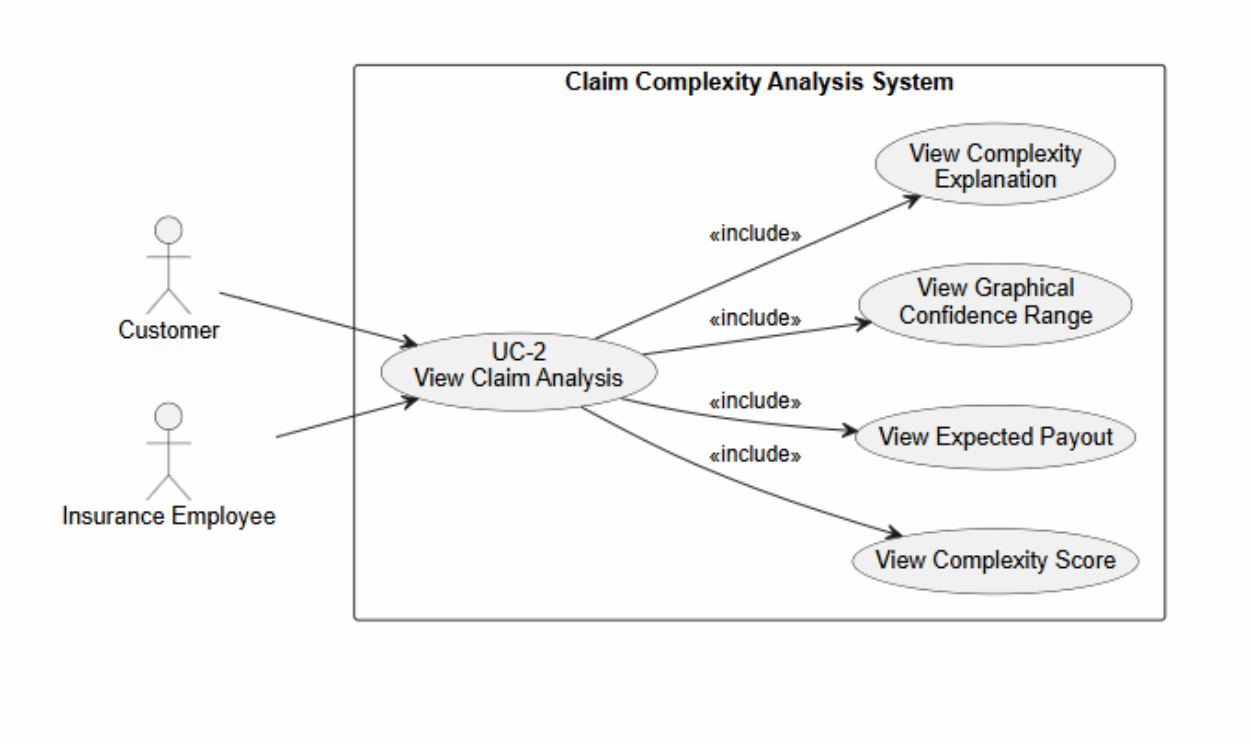



**Figure 2.** UC-2 UML diagram

**UC-2: View Claim Analysis**

**Actor:** Customer or Insurance Employee

The user views the complexity score, expected payout, graphical confidence range, and an explanation of the factors contributing to the complexity score.

**Related Requirements:** FR-2, FR-3, FR-4, FR-5, NFR-2

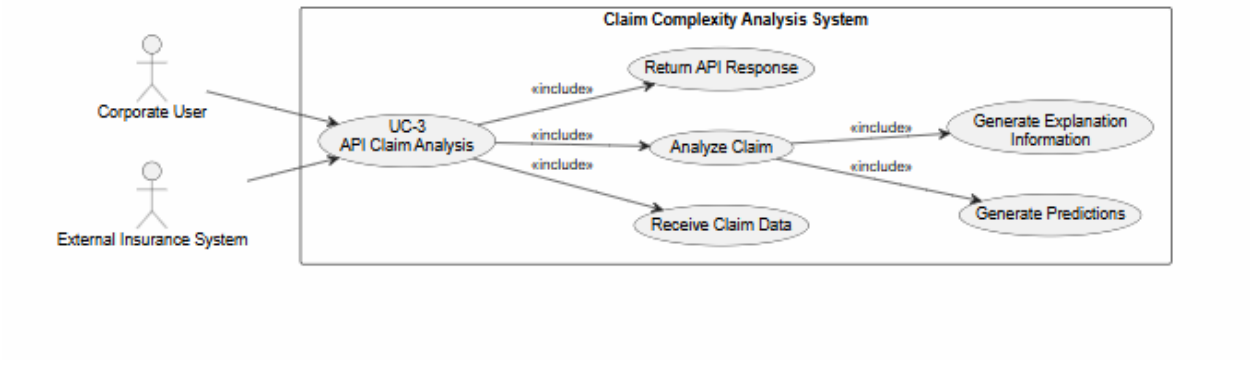



**Figure 3.** UC-3 UML diagram

**UC-3: API Claim Analysis**

**Actor:** Corporate User / External System

An external insurance system submits claim information through the API and receives the model's predictions and applicable explanation information.

**Related Requirements:** FR-6, NFR-3

---

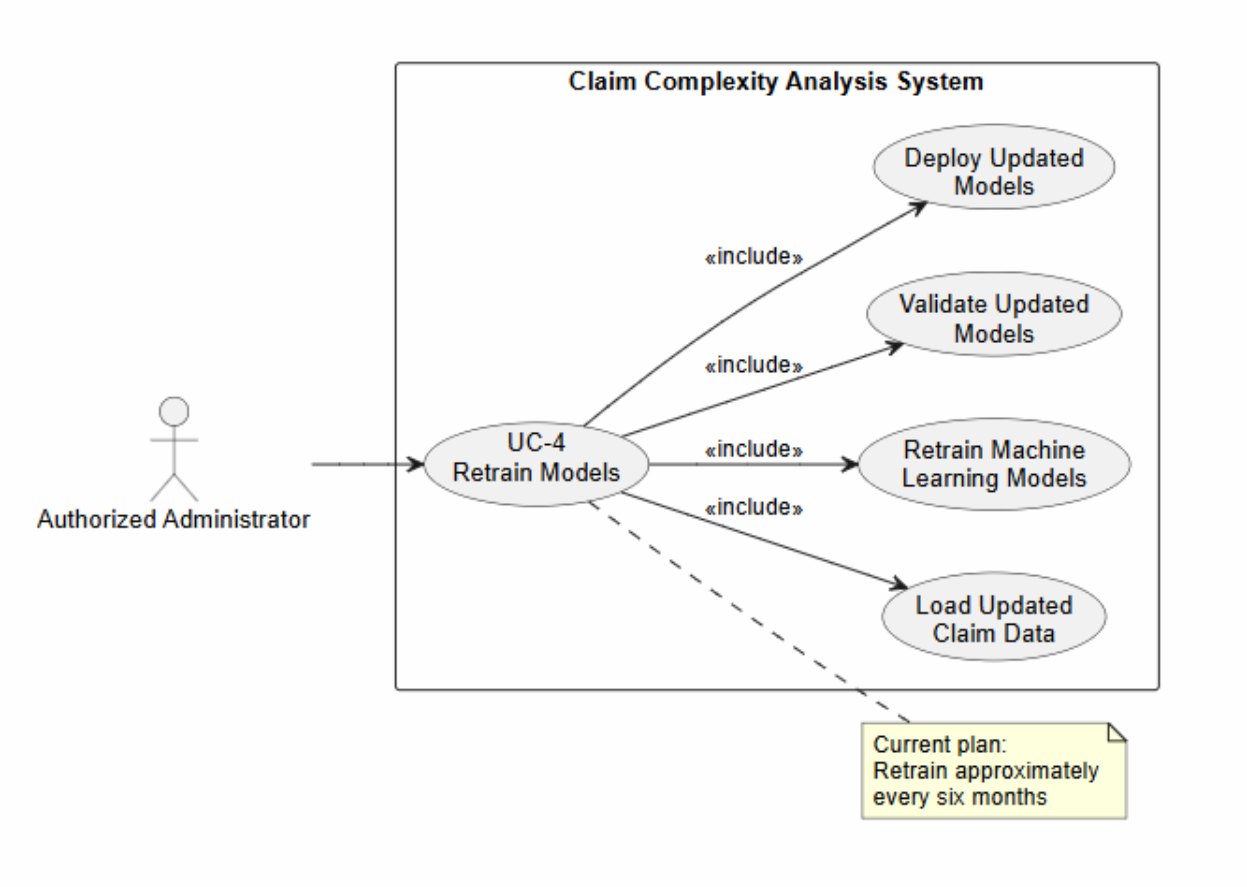



**Figure 4.** UC-4 UML diagram

**UC-4: Retrain Models**

**Actor:** Authorized Administrator

The system's machine learning models are retrained using updated claim data. The current plan is to retrain approximately every six months.

**Related Requirements:** FR-7, NFR-4

### III.2. Functional Requirements

#### III.2.1. Claim Analysis and Complexity Model

**FR-1 — Claim Data Processing:** The system must accept and process insurance claim information for training and inference. The initial dataset is expected to contain approximately 10,000 records covering five years and approximately 7 MB of data.

**Source:** Project stakeholders need to analyze historical and new insurance claims.
**Priority:** Level 0

**FR-2 — Complexity Score:** The system must produce a claim complexity score representing the expected difficulty of processing a claim. The complexity score must be calculated as a weighted average of multiple sub-scores.

The expected sub-scores include factors such as:

- Litigation chance
- Medical involvement/chance
- Return-to-work status
- Rehabilitation
- Other complications

[TBD – Sub scores, weight, fields are TBD]

> **Source:** Project stakeholders; need to estimate claim difficulty using multiple contributing factors.
> **Priority:** Level 0

**FR-3 — Complexity Sub-Scores:** The system must calculate the individual sub-scores used to produce the overall complexity score. The sub-scores must be combined according to configurable weights. The sub-scores are expected to cover factors such as time loss, medical conditions, return to work, vocational rehabilitation, litigation, permanent partial disability, settlement, and total claim cost. Examiner touch points must not be used on their own to decide that a claim is complex because a high number of touch points can come from how an examiner works rather than from the claim itself. The final sub-scores and their starting weights will be decided after the team reviews the claims data, using closed claims with known final costs as a reference.

**Source:** Project stakeholders; need to represent different sources of claim complexity.
**Priority:** Level 0

**FR-4 — Complexity Explanation:** The system must provide an explanation of the complexity score that identifies the factors contributing to the score. The explanation should communicate which sub-scores and other claim characteristics have the greatest effect on the overall result. Factors that come from the sanitized diary notes may be mapped back to readable categories so they can be understood in the explanation. The exact layout of the explanation will be decided during the design phase.

**Source:** Penser examiners, leadership, and engineering need to understand why the system produced a particular complexity score.
**Priority:** Level 0

**FR-5 — Expected Cost Estimate:** The system must produce an expected cost estimate for an analyzed claim based on historical claim information and relevant claim characteristics. The cost estimate must be produced alongside the complexity score for every claim. Closed claims with known final costs must be used as the reference for these estimates.

**Source:** Project stakeholders; need to estimate potential claim cost.
**Priority:** Level 0

**FR-6 — Confidence Range:** The system must display prediction confidence as a graphical range for the cost estimate and other applicable predictions. For examiners, confidence must also be shown as a percentage from 0 to 100 to stay consistent with other platforms Penser uses. Leadership and engineering must be able to view the graphical range. The method used to calculate confidence will be decided during the design phase.

**Source:** Project stakeholders; need to communicate prediction uncertainty.
**Priority:** Level 0

### III.2.2. User Interface and Integration

**FR-7 — Web Interface and API:** The system must provide a web interface for authenticated Penser examiners, leadership, and engineering, and an API that allows claim data to be sent to the system and results to be returned. The web interface must allow users to submit claims and view analysis results, including the complexity score, cost estimate, confidence, and complexity explanation. Access to both the web interface and the API must require authentication through AWS Cognito, which is part of the reference stack Penser specified. The specific API endpoints and data formats will be decided during the design phase.

**Source:** Penser examiners, leadership, and engineering need direct access to results, and claims.
**Priority:** Level 0

**FR-8 — Model Retraining:** The system must support periodic model retraining using new available claim data. The current target is about every six months. This schedule may be reevaluated later to account for inflation and changes in new arriving claims. If accuracy is too low or the data lacks coverage, the training data may be expanded to include older claims going back as far as the 1990s. When retraining is ready, the team will give Penser a proposed approach and estimated cost so the budget can be reviewed. The model approach will be chosen after the team reviews the available data during the planning and research sprint.

**Source:** Project stakeholders; need to maintain model relevance as new data becomes available.
**Priority:** Level 1

**FR-9 — Inflation Adjustment:** The system must account for changes in monetary values when using historical claim costs. The current approach is to use weighted

inflation adjustments. The inflation weight must be tunable along with the other weights. This adjustment also needs to account for historical cost differences if older claims going back to the 1990s are added to the data.

**Source:** Project stakeholders; need for meaningful cost estimates across different time periods.
**Priority:** Level 1

### III.3. Non-Functional Requirements

**NFR-1 — Performance:** The system shall support approximately one claim per minute under normal usage and up to three claims per minute under higher usage. The maximum expected inference time is 10 minutes.

**NFR-2 — Usability and Explainability:** The system shall present the complexity score, expected payout, graphical confidence range, and complexity explanation clearly to customers and insurance employees.

**NFR-3 — Security:** The system shall require authentication for Penser examiners, leadership, and engineering. The system should only use diary notes that Penser has sanitized, with identifying information removed and dictionary words tokenized. The authentication technology, user permissions, and API security will be decided during the design phase.

**NFR-4 — Scalability:** The system shall be capable of future integration into larger insurance systems through its API.

**NFR-5 — Cost:** The system shall be developed within an expected total budget of approximately $300–$500, including approximately $100–$300 in AWS credits.

**NFR-6 — Maintainability:** The system should separate the machine learning models, web interface, and API sufficiently to allow individual components to be updated. The complexity score weights should be configurable without requiring major changes to the system. Development should begin locally, and AWS or SageMaker will be evaluated later for hosting so the system can move to the cloud. The exact deployment, infrastructure, and testing requirements will be decided during the design phase.

### III.4.1 User Stories

**US-1 — Customer Submits a Claim**

**User Story:**
As a customer, I want to submit my claim information, so that the system can analyze my claim.

**Success Conditions:**
**Given** I am an authenticated customer with valid claim information,

**When** I submit my claim,

**Then** the system accepts the claim and generates a complexity score, expected payout, confidence range, and explanation of the complexity score.

**Exceptional Conditions:**
**Given** my claim information is incomplete or invalid,

**When** I submit the claim,

**Then** the system informs me that the claim cannot be processed and identifies the required information.

**Related Requirements:** FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, NFR-1, NFR-3

### US-2 — Customer Views Claim Analysis

**User Story:**
As a customer, I want to view my claim analysis, so that I can understand the expected complexity and cost of my claim.

**Success Conditions:**
**Given** my claim has been successfully analyzed,

**When** I view the claim results,

**Then** the system displays the complexity score, contributing factors, expected payout, graphical confidence range, and explanation of the complexity score.

**Exceptional Conditions:**
**Given** an analysis is unavailable or cannot be completed,

**When** I attempt to view the results,

**Then** the system informs me that the analysis is unavailable.

**Related Requirements:** FR-2, FR-3, FR-4, FR-5, FR-6, NFR-2

### US-3 — Claims Handler Analyzes a Claim

**User Story:**
As a claims handler, I want to analyze a claim, so that I can better understand its complexity and expected cost.

**Success Conditions:**
**Given** I am an authenticated claims handler with valid claim information,

**When** I submit the claim for analysis,

**Then** the system provides the complexity score, contributing sub-scores, expected payout, confidence range, and explanation of the complexity score.

**Exceptional Conditions:**
**Given** the claim information is incomplete or invalid,

**When** I submit the claim,

**Then** the system identifies the invalid or missing information and does not generate an analysis.

**Related Requirements:** FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, NFR-1, NFR-3

### US-4 — Corporate User Integrates the System

**User Story:**
As an insurance company, I want to access the claim analysis system through an API, so that I can integrate claim analysis into existing software.

**Success Conditions:**
**Given** the external system provides valid claim information through the API,

**When** the API receives the request,

**Then** the system processes the claim and returns the applicable analysis results, including the complexity score, expected payout, and explanation information.

**Exceptional Conditions:**
**Given** the API request contains invalid or incomplete information,

**When** the request is received,

**Then** the system returns an error indicating that the request cannot be processed.

**Related Requirements:** FR-7, NFR-1, NFR-4

### US-5 — Administrator Retrains the Model

**User Story:**
As an administrator, I want to retrain the machine learning models, so that the system can incorporate newly available claim data.

**Success Conditions:**
**Given** updated claim data is available,

**When** the administrator initiates model retraining,

**Then** the system trains the models using the updated data and makes the updated models available for future analysis.

**Exceptional Conditions:**
**Given** the training data is invalid or unavailable,

**When** the administrator attempts to retrain the models,

**Then** the system reports the problem and does not replace the existing models.

**Related Requirements:** FR-8, NFR-6

### III.4.2. Traceability Matrix

| Stakeholder Need | User Story | Functional Requirement | Non-Functional Requirement | Priority |
|---|---|---|---|---|
| Analyze claims | US-1, US-3 | FR-1 | NFR-1 | Level 0 |
| Measure claim complexity | US-2, US-3 | FR-2, FR-3 | NFR-2 | Level 0 |
| Explain claim complexity | US-2, US-3 | FR-4 | NFR-2 | Level 0 |
| Estimate claim cost | US-2, US-3 | FR-5 | NFR-2 | Level 0 |
| Understand prediction uncertainty | US-2, US-3 | FR-6 | NFR-2 | Level 0 |
| Integrate with insurance systems | US-4 | FR-7 | NFR-4 | Level 0 |
| Maintain model relevance | US-5 | FR-8 | NFR-6 | Level 1 |
| Account for changing costs | US-2, US-3 | FR-9 | NFR-6 | Level 1 |

### III.5. Standards

The system will follow established software engineering, accessibility, artificial intelligence, and professional ethics standards to support reliable, maintainable, and responsible development. IEEE 830-1998, Recommended Practice for Software Requirements Specifications, is used as a guideline for structuring and documenting the system requirements. The standard supports the organization of the project's functional requirements, non-functional requirements, use cases, and traceability between stakeholder needs and system requirements [1].

The system's web interface will follow W3C Web Content Accessibility Guidelines (WCAG) 2.2, with the project targeting Level AA where applicable [2]. Accessibility considerations will apply to the presentation of claim complexity scores, expected payout estimates, confidence ranges, and complexity explanations. The interface will use readable text, sufficient color contrast, clear labels, and keyboard-accessible controls. Accessibility will be evaluated during interface testing before deployment.

Because the system uses machine learning to analyze insurance claims, the project will follow the principles of the NIST Artificial Intelligence Risk Management Framework (AI RMF 1.0) [3]. The framework is relevant to the project's requirements for model reliability, transparency, explainability, and risk management. In particular, the system will provide an explanation of the factors contributing to a claim's complexity score rather than presenting the score as an unexplained prediction. Model performance will also be evaluated using held-out data during development and retraining. Specific fairness metrics and thresholds will be determined during model development.

The project will also follow the ACM Code of Ethics and Professional Conduct when designing and implementing the system [4]. This is particularly relevant because insurance claims can contain sensitive information and the system's predictions could influence how claims are handled. The system is therefore intended as a decision-support tool rather than a replacement for human judgment. Access to claim information will require authentication, and users will only be provided access appropriate to their role. The team will also evaluate the model for potential sources of bias as part of model validation.

For data security, the project will consider the NIST Cybersecurity Framework (CSF) 2.0 when designing authentication, access control, data protection, and API security [5]. Customers will only be permitted to access their own claims, while authorized insurance employees will be able to access claims appropriate to their role. Data transmitted between the web interface, API, and backend services will use HTTPS/TLS. Specific encryption-at-rest and data-retention configurations will be determined during implementation.

## IV. System Evolution

The initial system will use a defined machine learning architecture to generate claim analysis. The complexity component is expected to use multiple sub-scores representing different sources of claim complexity. These sub-scores will be combined using a weighted-average approach to produce the overall complexity score.

The exact sub-scores, weights, model implementation, and scoring scales remain under development. As additional claim data becomes available, the sub-score definitions and weights may be adjusted based on model evaluation and stakeholder feedback. The system is expected to be retrained approximately every six months.

Future versions may incorporate additional complexity factors, improved prediction models, additional outputs, and integration with larger insurance systems. The system should preserve the ability to update individual components without requiring a complete redesign.

The machine learning approach has not been chosen yet. The team will select it during the planning and research sprint after reviewing the column headers and data provided by Penser.

## V. Glossary

**Claim Complexity Score:** A numerical estimate representing the expected difficulty of processing an insurance claim. The score is expected to be calculated as a weighted average of multiple complexity sub-scores.

**Complexity Sub-Score:** A score representing an individual factor contributing to claim complexity, such as litigation chance, medical involvement, return to work, rehabilitation, or other complications.

**Expected Payout:** The system's estimate of the potential financial cost associated with an insurance claim.

**Confidence Range:** A graphical representation of the uncertainty associated with a model prediction.

**Complexity Explanation:** Information describing the factors and sub-scores that contributed to the overall complexity score.

**Inference:** Using a trained machine learning model to produce a prediction for new claim data.

**Retraining:** Updating a machine learning model using additional or newer training data.

**API:** An interface that allows external software systems to communicate with the claim analysis system.

### VI. References

[1] Institute of Electrical and Electronics Engineers, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[2] World Wide Web Consortium (W3C), *Web Content Accessibility Guidelines (WCAG) 2.2*, W3C Recommendation, 2024.

[3] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, National Institute of Standards and Technology, 2023.

[4] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, ACM, 2018.

[5] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST Cybersecurity White Paper 29, 2024.